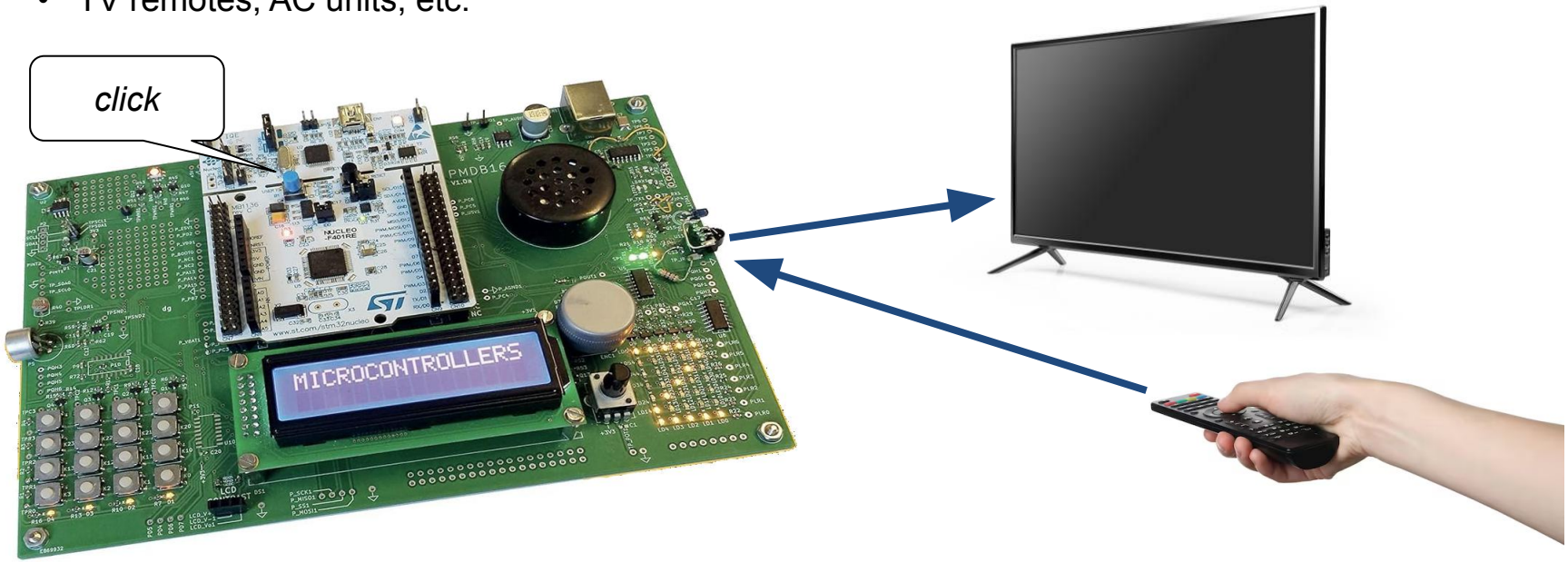
POLITECNICO
MILANO 1863

Memorisation and replay of infrared remotes

Federico Bocchieri

Project description

- This project implements a tool to **sample and replay IR signals**
- TV remotes, AC units, etc.



Project description

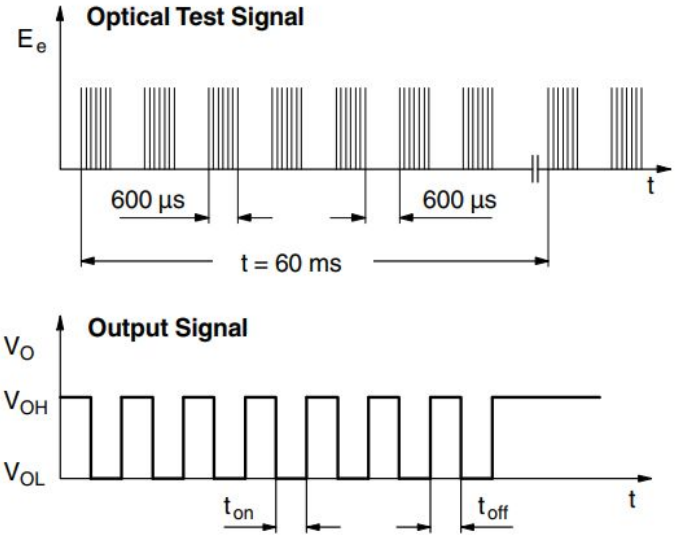
- It runs on **Miosix 3.0x** on an **STM32 Nucleo board** + **PMDB16 evaluation board**

Infrared transmission

- A sequence of **zeroes** and **ones**, **with a catch**
- How to **filter out visible light**?
- How about **other infrared sources** e.g. *the sun*?

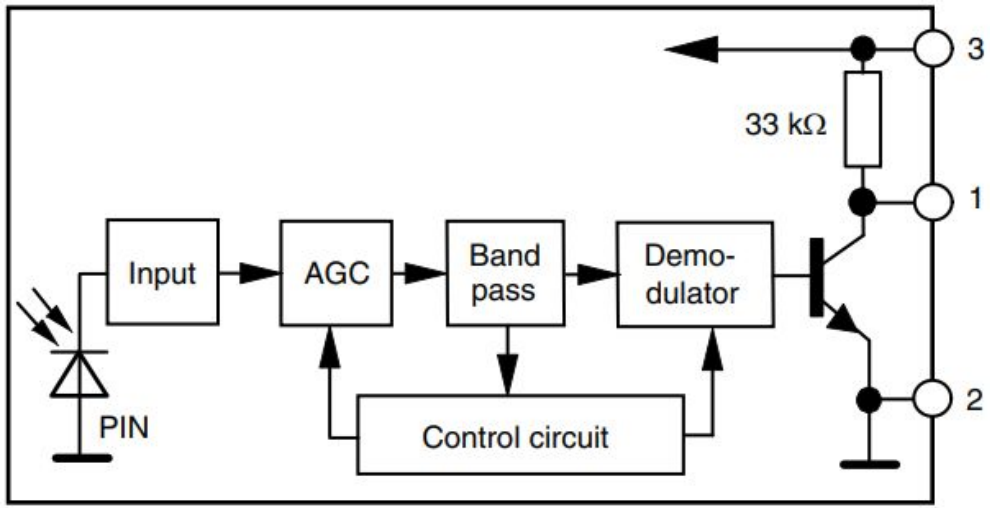
Sending data

- LED is **modulated** at 38KHz (typically)
- **Active levels** are **encoded as 38KHz pulses**



Receiving data

- Received light is **demodulated** and converted back into **sequence of zeroes and ones**
- **IR receivers typically integrate the required circuitry**



In practice

Sending data:

- Generate a **free-running 38KHz 50% DC wave** using a **timer in PWM mode**
- Connect **timer to GPIO pin** of the IR LED
- **Toggle timer channel on and off** to send data

Receiving data:

- The signal is **already demodulated** (i.e. a digital signal already)
- **Each time the signal changes**, save its **level** and **timestamp** of occurrence somewhere
- Get the timestamp from a **high resolution timer**
- *Reasonable assumption**: signal does not last very long
  - **Stop recording after some time**

Receiving data - in practice

- Configure **interrupt on signal edge** to sample data

```
// Configure IR receiver pin with interrupt on edge change
IR_Receiver::mode(Mode::INPUT); // Configure IR receiver pin as input
SYSCFG->EXTICR[2] = SYSCFG_EXTICR3_EXTI10_PA;   // Route IR pin (PA10) to EXTI10 line
EXTI->RTSR |= EXTI_RTSR_TR10;   // Trigger interrupt on rising edge
EXTI->FTSR |= EXTI_FTSR_TR10;   // Trigger interrupt on falling edge
IRQregisterIrq(lock, EXTI15_10_IRQn, &PMDB_IR::isr_sample, this);   // Register ISR to sample
```

Receiving data - in practice

- Configure timer as **one-pulse 1 second timer**, with **1us resolution**
- Configure **interrupt on timer elapse** to stop timer

```
GlobalIrqLock lock;
reset_TIM2();

RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;      // Enable timer clock
TIM2->CR1 |= TIM_CR1_OPM;                // One-pulse mode (stops timer at the next update event
TIM2->PSC = 84000000 / 1000000 - 1;      // Configure 1MHz timer (i.e. 1us resolution).
TIM2->ARR = 1000000 - 1;                 // Count 1000000us == 1s
TIM2->EGR |= TIM_EGR_UG;                 // Generate update event to update register values
TIM2->SR &= ~TIM_SR_UIF;                 // Clear update flag caused by setting UG
IRQregisterIrq(lock, TIM2_IRQn,
    &PMDB_IR::isr_stop_sampling, this); // Register ISR to stop sampling
```

Receiving data - in practice

- Inside **signal edge interrupt**, start timer (if not running) and **sample level** and **timestamp**

```
// Check that interrupt is pending for EXTI10
if (!(EXTI->PR & EXTI_PR_PR10)) {
    return;
}
EXTI->PR |= EXTI_PR_PR10;   // Clear interrupt pending bit

// Start timer if not started already
if (!(TIM2->CR1 & TIM_CR1_CEN)) {   // Note: in one-pulse mode, CEN bit is cleared automatically
    TIM2->CR1 |= TIM_CR1_CEN;       // Start timer
}

// Sample timestamp and IR port level
bool signal_level = IR_Receiver::value();
uint timestamp_us = TIM2->CNT;
buffer.push_back({signal_level, timestamp_us}); // Note: interrupt is triggered on signal edge,
```

Receiving data - in practice

- Inside **timer elapse interrupt**, disable (unmask) both interrupts to stop recording

```
// Disable all interrupts to stop recording, then wake up the receiver function
if (TIM2->SR & TIM_SR_UIF) {
    TIM2->SR &= ~TIM_SR_UIF;        // Clear update flag
    EXTI->IMR &= ~EXTI_IMR_IM10;     // Disable edge triggered interrupt on GPIO pin
    TIM2->DIER &= ~TIM_DIER_UIE;    // Disable this interrupt (timer update pin on overflow)

    // Wake up receiver function
    is_recording->IRQwakeup();    // Will occur as soon as going out of the interrupt
    is_recording = nullptr;
}
```

Sending data - in practice

- Configure **LED pin** to be driven by **timer in PWM mode**

```
// Configure IR LED pin to be driven by TIM2 CH3 in PWM mode
IR_LED::mode(Mode::ALTERNATE);
IR_LED::alternateFunction(1);    // TIM2 CH3 (PWM) output
```

Sending data - in practice

- Configure **timer** as **PWM mode** to generate 38KHz carrier

```
RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;      // Enable clock to TIM2
TIM2->CR1 |= TIM_CR1_ARPE;               // Enable auto-reload preload for timer
TIM2->CCMR2 |= TIM_CCMR2_OC3M_2
    | TIM_CCMR2_OC3M_1;                  // Enable PWM mode 1 (110) for channel 3
TIM2->CCMR2 |= TIM_CCMR2_OC3PE;          // Enable auto-reload preload for channel 3
TIM2->PSC = 0;                           // No prescaling, keep time base at 84MHz
TIM2->ARR = 84000 / 38 - 1;              // Set frequency to 38KHz
TIM2->CCR3 = (84000 / 38 - 1) / 2;       // Set DC to 50%
TIM2->CNT=0;                             // Reset counter to 0
TIM2->EGR |= TIM_EGR_UG;                 // Reinitialize counter and update registers
TIM2->CR1 |= TIM_CR1_CEN;                // Start time base (notably, the timer keeps running
TIM2->CCER &= ~TIM_CCER_CC3E;            // Make sure channel output is disabled for now
```

Sending data - in practice

- To send data, **busy wait until next timestamp** and **switch PWM channel** on and off

```
for (auto& sample: buffer) {

    int next_ts_us = start_ts_us + sample.timestamp_us;
    while (IRQgetTime() / 1000 - next_ts_us < 0) ;  // Busy wait until system timer exceeds
                                                    // the current sample's timestamp

    // Toggle PWM channel according to signal edge
    if (sample.level)                    // Rising edge
        TIM2->CCER &= ~TIM_CCER_CC3E;    // Disable timer channel
    else                                 // Falling edge -> stimulate receiver (active low)
        TIM2->CCER |= TIM_CCER_CC3E;     // Enable timer channel
}
```

Testing

- Compare **VCD dumps**
  - From logic analyzer
  - Produced by my class

```
void pprint_vcd() {
    const uint shift_us = 10000;
    iprintf("$version PMDB16 IR dumper $end\n");
    iprintf("$timescale 1 us $end\n");
    iprintf("$scope module top $end\n");
    iprintf("$var wire 1 ! ir $end\n");
    iprintf("$upscope $end\n");
    iprintf("$enddefinitions $end\n");
    iprintf("#0 1!\n");                          // "fake" rising edge
    for (auto& sample: buffer) {
        iprintf("#%d %d!\n", sample.timestamp_us + shift_us, sample.level);
    }
}
```

Testing

- In-situ approach
  - It Just Works™ *[Citation needed]*

Outcomes - deliverables

- `main.cpp` with logic to **start recording / sending IR data, toggle modes and print status text**
- `PMDB_IR` class implementing **all aforementioned logic**
- A few **waveform dumps**
- Python script to **convert VCDs to my internal representation**
- Project **report**
- This **presentation**

Learning outcomes

- Learning how to **navigate complex codebases**
  - Miosix - to learn from more experienced people
- Learning to **read complex technical documentation**
  - ST Nucleo documentation
- **Low-level microcontroller device configuration**
  - Writing device registers
- How to **sample digital signals**
  - Oversampling, run-length encode, etc.
- Internal **RTOS code structure**
- Using a **logic analyzer**

Problems I encountered

- **Repurpose timer**
  - For **sending**, apparently IR LED pin has **only TIM2 CH3 as timer alternate function**
  - For **sampling** I needed a 32bit timer, and **TIM5 was already taken…**
  - **Timer reconfiguration**!

- Doing this with no tools is tough
  - **Out of reach of a logic analyzer** for two whole days
  - No way to **cross check**